



Section 2 / 24

Computer Architecture

general description for main components in computer architecture

Today, most modern computers are built on what is known as the **Von Neumann Architecture**, which was developed back in 1945 by **Von Neumann** to enable the creation of "General-Purpose Computers" as **Alan Turing** described them at the time. **Alan Turing** in turn, based his ideas on **Charles Babbage**'s mid-19th century "Programmable Computer" concept. Note that all of these people were mathematicians.

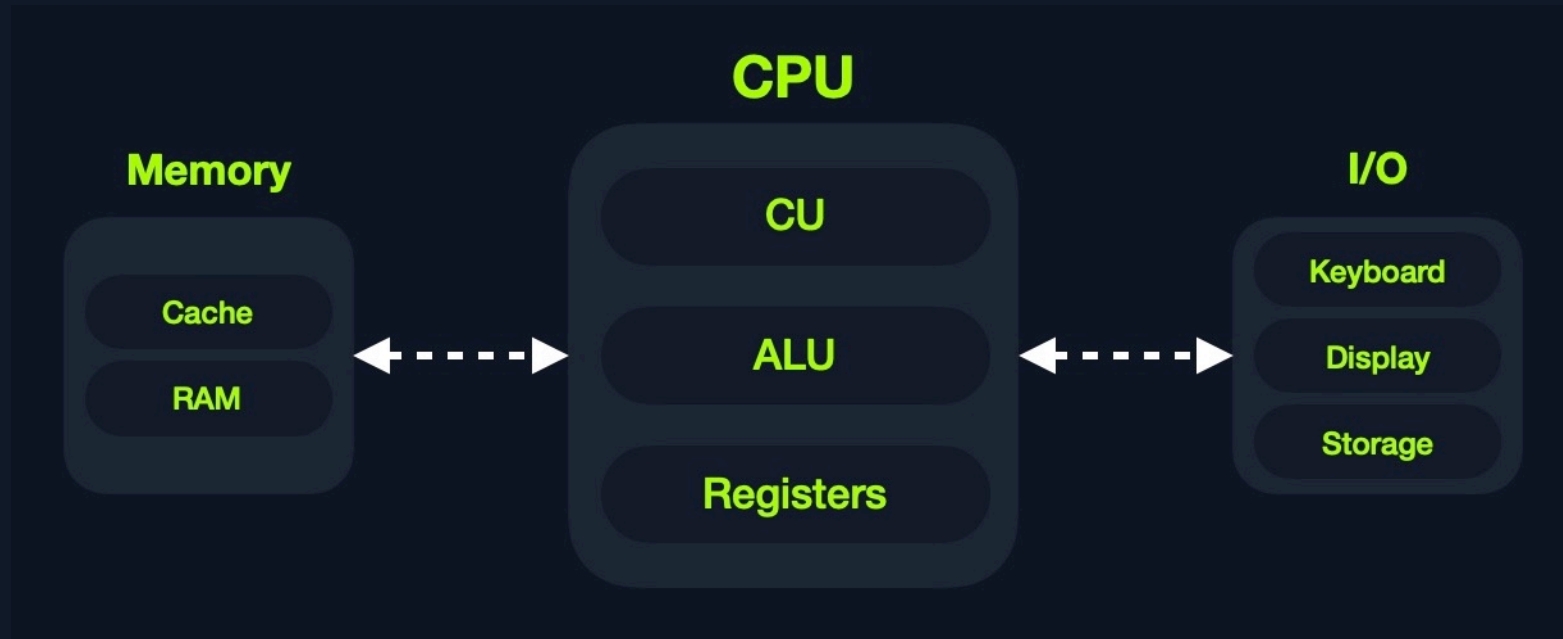
This architecture executes machine code to perform specific algorithms. It mainly consists of the following elements:

- Central Processing Unit (CPU)
- Memory Unit
- Input/Output Devices
 - Mass Storage Unit
 - Keyboard
 - Display

Furthermore, the CPU itself consists of three main components:

- Control Unit (CU)

- Arithmetic/Logic Unit (ALU)
- Registers



Though very old, this architecture is still the basis of most modern computers, servers, and even smartphones.

Assembly languages mainly work with the CPU and memory. This is why it is crucial to understand the general design of computer architecture, so when we start using assembly instructions to move and process data, we know where it's going and coming from and how fast/expensive each instruction is.

Furthermore, basic and advanced binary exploitation requires a proper understanding of computer architecture. With basic stack overflows, we only need to be aware of the general design. Once we start using ROP and Heap exploits, our understanding should be profound. Let us now take a deeper look into some essential components.

Memory

A computer's memory is where the **temporary** data and instructions of currently running programs are located. A computer's memory is also known as Primary Memory. It is the primary location the CPU uses to retrieve and process data. It does so very frequently (billions of times a second), so the memory must be extremely fast in storing and retrieving data and instructions.

There are two main types of memory:

1. **Cache**
2. **Random Access Memory (RAM)**

Cache

Cache memory is usually located within the CPU itself and hence is extremely fast compared to RAM, as it runs at the same clock speed as the CPU. However, it is very limited in size and very sophisticated, and expensive to manufacture due to it being so close to the CPU core.

Since RAM clock speed is usually much slower than the CPU cores, in addition to it being far from the CPU, if a CPU had to wait for the RAM to retrieve each instruction, it would effectively be running at much lower clock speeds. This is the main benefit of cache memory. It enables the CPU to access the upcoming instructions and data quicker than retrieving them from RAM.

There are usually three levels of cache memory, depending on their closeness to the CPU core:

Level	Description
Level 1 Cache	Usually in kilobytes, the fastest memory available, located in each CPU core. (Only registers are faster.)

Level	Description
Level 2 Cache	Usually in megabytes, extremely fast (but slower than L1), dedicated to each CPU core. Serves as a middle tier between L1 and L3.
Level 3 Cache	Usually in megabytes (larger than L2), faster than RAM but slower than L1/L2. (Not all CPUs use L3.)

RAM

RAM is much larger than cache memory, coming in sizes ranging from gigabytes up to terabytes. RAM is also located far away from the CPU cores and is much slower than cache memory. Accessing data from RAM addresses takes many more instructions.

For example, retrieving an instruction from the registers takes only one clock cycle, and retrieving it from the L1 cache takes a few cycles, while retrieving it from RAM takes around 200 cycles. When this is done billions of times a second, it makes a massive difference in the overall execution speed.

In the past, with 32-bit addresses, memory addresses were limited from `0x00000000` to `0xffffffff`. This meant that the maximum possible RAM size was 2^{32} bytes, which is only 4 gigabytes, at which point we run out of unique addresses. With 64-bit addresses, the range is now up to `0xffffffffffffffff`, with a theoretical maximum RAM size of 2^{64} bytes, which is around 18.5 exabytes (18.5 million terabytes), so we shouldn't be running out of memory addresses anytime soon.

When a program is run, all of its data and instructions are moved from the storage unit to the RAM to be accessed when needed by the CPU. This happens because accessing them from the

storage unit is much slower and will increase data processing times. When a program is closed, its data is removed or made available to re-use from the RAM.

As we can see, the RAM is split into four main `segments` :

Segment	Description
<code>Stack</code>	Has a Last-in First-out (LIFO) design and is fixed in size. Data in it can only be accessed in a specific order by push-ing and pop-ing data.
<code>Heap</code>	Has a hierarchical design and is therefore much larger and more versatile in storing data, as data can be stored and retrieved in any order. However, this makes the heap slower than the Stack.
<code>Data</code>	Has two parts: <code>Data</code> , which is used to hold variables, and <code>.bss</code> , which is used to hold unassigned variables (i.e., buffer memory for later allocation).
<code>Text</code>	Main assembly instructions are loaded into this segment to be fetched and executed by the CPU.

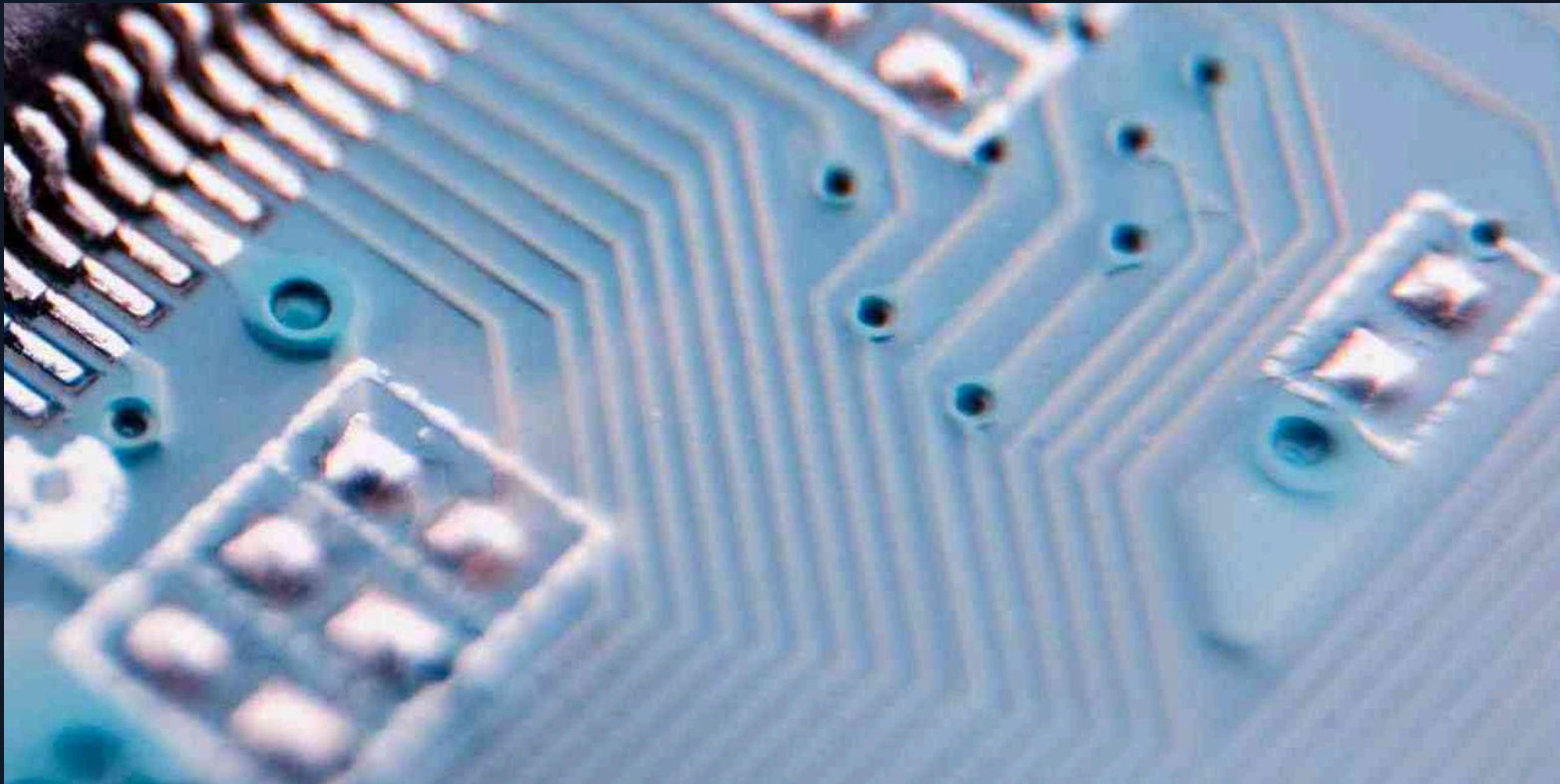


Although this segmentation applies to the entire RAM, `each application is allocated its Virtual Memory when it is run`. This means that each application would have its own `stack`, `heap`, `data`, and `text` segments.

IO/Storage

Finally, we have the Input/Output devices, like the keyboard, the screen, or the long-term storage unit, also known as Secondary Memory. The processor can access and control IO devices using **Bus Interfaces**, which act as 'highways' to transfer data and addresses, using electrical charges for binary data.

Each Bus has a capacity of bits (or electrical charges) it can carry simultaneously. This usually is a multiple of 4-bits, ranging up to 128-bits. Bus interfaces are also usually used to access memory and other components outside the CPU itself. If we take a closer look at a CPU or a motherboard, we can see the bus interfaces all over them:



Unlike primary memory that is volatile and stores temporary data and instructions as the programs are running, the storage unit stores permanent data, like the operating system files or entire

applications and their data.

The storage unit is the slowest to access. First, because they are the farthest away from the CPU, accessing them through bus interfaces like SATA or USB takes much longer to store and retrieve the data. They are also slower in their design to allow more data storage. As long as there is more data to go through, they will be slower.

There has been a shift from classic magnetic storage units, like tapes or Hard Disk Drives (HDD), to Solid-State Drives (SSD) in recent years. This is because SSD's utilize a similar design to RAM's, using non-volatile circuitry that retains data even without electricity. This made storage units much faster in storing and retrieving data. Still, since they are far away from the CPU and connected through special interfaces, they are the slowest unit to access.

Speed

As we can see from the above, the further away a component is from the CPU core, the slower it is. Also, the more data it can hold, the slower it is, as it simply has to go through more to fetch the data. The below table summarizes each component, its size, and its speed:

Component	Speed	Size
Registers	Fastest	Bytes
L1 Cache	Fastest, other than Registers	Kilobytes
L2 Cache	Very fast	Megabytes

Component	Speed	Size
L3 Cache	Fast, but slower than the above	Megabytes
RAM	Much slower than all of the above	Gigabytes-Terabytes
Storage	Slowest	Terabytes and more

The speed here is relative depending on the CPU clock speed. Now that we have a general idea of the computer architecture, we'll discuss Registers and the CPU architecture in the next section.

[< Previous](#)

Section 2 / 24

 +20

[Next](#)[Mark Complete & Next](#)